



Università degli Studi
Guglielmo Marconi

UNIVERSITÀ DEGLI STUDI GUGLIELMO MARCONI

DIPARTIMENTO DI SCIENZE INGEGNERISTICHE

CORSO DI LAUREA IN INGEGNERIA INFORMATICA

*“Robustezza di classificatori neurali in condizioni di informazione
incompleta: riduzione dei campioni, rimozione di classi e recupero
few-shot”*

Relatore:

Prof. Aldegheri Stefano

Candidato:

Gargiulo Umberto

Indice

1	Introduzione e definizione del problema	4
1.1	Analisi del problema: Classificatori flat vs gerarchici	4
1.2	Obiettivo e impostazione dello studio	5
1.3	Struttura della tesi	6
2	Fondamenti teorici sulle architetture	7
2.1	Reti neurali convoluzionali e principi di funzionamento	7
2.2	Backbone per l'estrazione di feature	9
2.2.1	ResNet	9
2.2.2	DenseNet	11
2.2.3	EfficientNet	12
2.3	Analisi comparativa	13
3	Pipeline sperimentale e metodologia	14
3.1	Dataset e task	14
3.2	Architettura multi-task	14
3.3	Procedura di training dei modelli	15
3.3.1	Regime di addestramento	15
3.3.2	Ottimizzazione	16
3.3.3	Trasformazioni sui dati	16
3.3.4	Early stopping	16
3.4	Degradazione controllata del training set	17
3.5	Metriche di valutazione	18
4	Risultati e discussione	20
4.1	Risultati sul dataset Oxford-IIIT Pet	20

4.2	Risultati sul dataset CIFAR-100	25
4.3	Discussione sui risultati ottenuti	28
5	Approccio basato su reti prototipiche	30
5.1	Motivazione: il problema delle classi non viste	30
5.2	Reti prototipiche: principio di funzionamento	30
5.3	Setup e campionamento	31
5.4	Risultati ottenuti e discussione	31
6	Conclusioni e sviluppi futuri	34
6.1	Sintesi e limiti dello studio	34
6.2	Sviluppi futuri	35
6.3	Considerazioni finali	35
	Bibliografia	37

Capitolo 1

Introduzione e definizione del problema

1.1 Analisi del problema: Classificatori flat vs gerarchici

Il presente elaborato indaga la robustezza di classificatori gerarchici a due livelli, ove il primo livello specifica la superclasse di appartenenza e il secondo specifica la sottoclasse.

Tali modelli operano contemporaneamente a livello generale, definito *coarse*, e a livello specifico di sottoclasse, definito *fine*. La classificazione *coarse* richiede l'estrazione di feature semantiche globali. La classificazione *fine* è intrinsecamente più complessa: il modello deve individuare dettagli visivi altamente specifici e discriminativi tra soggetti affini, appartenenti allo stesso gruppo generale.

Un classificatore *flat*, diversamente da un classificatore gerarchico a due livelli, assegna l'immagine a una delle possibili N classi disponibili senza sfruttare somiglianze o differenze nette tra elementi di classi diverse. Ogni classe è trattata come equidistante dalle altre, indipendentemente dal fatto che alcune categorie condividano caratteristiche visive affini. Per un classificatore *flat*, confondere un *labrador* e un *golden retriever* costituisce un errore equivalente a confondere lo stesso *labrador* con un *merluzzo*.

Queste somiglianze sono sfruttate dai classificatori gerarchici a due livelli. Dal punto di vista architetturale, il modello diventa multi-task: un backbone condiviso estrae feature come bordi, texture e forme, mentre due head distinte, che utilizzano tali feature come base di partenza, si specializzano una nella classificazione *coarse* e l'altra in quella *fine*.

Si consideri, a titolo esemplificativo, un dataset con due superclassi, *Piante* e *Animali*, articolate a loro volta in *Arboree* ed *Erbacee* le prime, e in *Mammiferi* e *Pesci* i secondi.

La distinzione tra superclassi risulta un compito relativamente semplice, poiché esse presentano caratteristiche visive marcatamente diverse tra loro, basti immaginare un pino e un cane.

La classificazione delle sottoclassi, invece, espone la rete a un compito complesso.

È il caso di specie come i *cetacei*: pur condividendo caratteristiche fisiche dei *mammiferi*, il loro adattamento a un ambiente acquatico può indurre il classificatore a confonderli con i *pesci*.

Tale criticità nella classificazione costituisce il punto di partenza per l'analisi sperimentale presentata.

Per valutare la robustezza dei classificatori gerarchici, il presente studio introduce delle riduzioni controllate del dataset di addestramento, operando su due configurazioni distinte: riduzione uniforme dei campioni per classe (*macro-drop*) ed esclusione completa di singole classi (*micro-drop*).

1.2 Obiettivo e impostazione dello studio

L'obiettivo dello studio è analizzare la robustezza dei modelli in condizioni di disponibilità limitata di informazioni per l'addestramento.

Le due tipologie di drop sono introdotte a diversi livelli di intensità. Per ogni livello, viene valutato il comportamento della rete davanti a uno stesso test set, il quale è fisso per tutti i modelli.

L'analisi non si limita a valutare la capacità del modello di riconoscere categorie presenti nel set di addestramento, ma si estende al caso critico in cui alcune classi sono escluse totalmente dal set di training. In questa condizione, il classificatore non può riconoscere correttamente la classe, dato che tali classi sono escluse dal fine-tuning. È necessario un approccio diverso, che sfrutti il modello non come semplice classificatore, ma come estrattore di feature: a partire da poche immagini di supporto (*few-shot*) è possibile verificare se le rappresentazioni assimilate dalla rete siano sufficienti a discriminare e quindi a riconoscere categorie escluse dal fine-tuning.

La metodologia applicata e la pipeline adottata sono descritte nel dettaglio nei capitoli successivi.

1.3 Struttura della tesi

L'elaborato è organizzato come segue:

Il Capitolo 2 descrive brevemente le reti neurali convoluzionali e i diversi tipi di backbone utilizzati.

Il Capitolo 3 presenta la pipeline adottata, i dataset utilizzati, la definizione e l'implementazione delle due tipologie di drop (*macro-drop* e *micro-drop*) e le metriche di valutazione dei backbone.

Il Capitolo 4 presenta i risultati relativi all'impatto dei drop di tipologia *macro* e *micro*, analizzando l'effetto che questi provocano nei diversi backbone.

Il Capitolo 5 descrive e applica l'approccio delle reti prototipiche come soluzione alle basse prestazioni causate dal micro-drop.

Il Capitolo 6 conclude la tesi con una sintesi e delle considerazioni finali.

Capitolo 2

Fondamenti teorici sulle architetture

2.1 Reti neurali convoluzionali e principi di funzionamento

Le reti neurali convoluzionali (CNN, *Convolutional Neural Networks*) sono le architetture standard per l'elaborazione dei segnali visivi. Esse costituiscono un'evoluzione del *Multi-Layer Perceptron* grazie all'utilizzo di tensori multidimensionali ($H \times W \times C$), ove H indica l'altezza, W la larghezza e C il numero di canali. Tale rappresentazione permette di preservare informazioni come le relazioni spaziali tra pixel vicini durante l'elaborazione, attraverso tre principi fondamentali:

- L'uso dei campi ricettivi locali (*local receptive fields*);
- La condivisione dei pesi tra le diverse posizioni spaziali (*weight sharing*);
- L'equivarianza alla traslazione (se l'oggetto si sposta, anche la feature map si sposta).

Le reti raccolgono informazioni dall'immagine tramite l'utilizzo di filtri chiamati kernel. Questi analizzano una porzione locale di pixel adiacenti alla volta e producono un output associato a quella specifica posizione spaziale.

L'output è definito così:

- X : tensore di input;
- K : tensore dei pesi del kernel;
- r : raggio del kernel;
- C_{in} : numero di canali di input;
- b_{out} : bias associato al canale di uscita.

$$Y(i, j, c_{\text{out}}) = b_{\text{out}} + \sum_{c=1}^{C_{\text{in}}} \sum_{m=-r}^r \sum_{n=-r}^r X(i+m, j+n, c) K(m, n, c, c_{\text{out}}). \quad (2.1)$$

Ovvero, il valore di output Y nella posizione spaziale (i, j) e nel canale di uscita c_{out} è calcolato come somma pesata dei valori del tensore di input X in un intorno locale in posizione (i, j) di dimensione $(2r + 1) \times (2r + 1)$. Ad esempio, per $r = 1$ si ottiene un kernel di dimensione 3×3 , mentre per $r = 2$ si ottiene un kernel di dimensione 5×5 .

L'operazione viene eseguita considerando tutti i canali di input. Ogni valore di X viene moltiplicato per il corrispondente peso K , appreso durante la fase di addestramento, e i prodotti ottenuti vengono sommati tra loro. Infine, al risultato viene aggiunto il termine di bias b_{out} associato al canale di uscita.

Questa formulazione spiega i principi elencati precedentemente. Il valore in posizione (i, j) dipende solo dalla porzione di input relativa a quella posizione, e non dall'intera immagine come in un'architettura di rete MLP. Questo spiega il principio dei campi ricettivi locali: i pixel vicini sono più correlati tra loro e producono maggiori informazioni se analizzati insieme tramite i kernel.

Il principio della condivisione dei pesi permette l'utilizzo di un insieme di pesi K per calcolare Y in ogni posizione (i, j) del tensore di output. Senza di esso si avrebbe un insieme di pesi indipendenti per ogni posizione, ricadendo nel caso di layer localmente connessi (*locally connected*).

Dal principio della condivisione dei pesi si arriva al principio dell'equivarianza alla traslazione: lo stesso kernel K viene applicato a ogni posizione (i, j) del tensore di input. Un pattern individuato in (i, j) sarà rilevato allo stesso modo anche in $(i + \Delta, j + \Delta)$, qualora l'oggetto che lo produce si spostasse di una quantità Δ nell'immagine. Un semplice spostamento nell'input produce uno spostamento corrispondente nell'output, senza dover ricalibrare i pesi per riconoscerlo.

Ciò che si è appena descritto definisce una singola operazione di convoluzione. Nella pratica non è sufficiente a estrarre informazioni utili alla classificazione. Una CNN utilizza una sequenza di queste operazioni in quelli che vengono chiamati layer. Ogni layer riceve in input il tensore prodotto dal layer precedente. Il tensore viene elaborato e diventa input di un nuovo layer. I primi layer catturano informazioni semplici, come bordi e texture. I layer più profondi catturano pattern complessi e significativi.

Il numero di layer impilati si definisce profondità della rete.

Si può pensare che l'aumento della profondità permetta al modello di apprendere rappresentazioni sempre più complesse e astratte. Nelle architetture tradizionali prive di connessioni dirette (*plain networks*), tuttavia, l'aumento della profondità esaspera il problema del *vanishing gradient*.

Durante l'addestramento, la rete apprende aggiornando i propri pesi. Il gradiente della loss viene calcolato con la regola della catena, propagandosi all'indietro attraverso i layer della rete con la *backpropagation*. A ogni layer il gradiente viene moltiplicato per fattori spesso minori di 1. Con molti layer, il gradiente tende a ridursi esponenzialmente, fino a diventare trascurabile. Ciò provoca un aggiornamento dei pesi lento, nullo in casi estremi, causando un arresto dell'apprendimento.

Nelle reti prive di collegamenti diretti, una profondità eccessiva non migliora le prestazioni, ma le peggiora nettamente. Tale degrado non è causato dall'overfitting, ma da una specifica difficoltà di ottimizzazione. ResNet e DenseNet, utilizzate in questa tesi, nascono con strategie diverse per affrontare questo limite; EfficientNet si concentra invece sul ridimensionamento efficiente dell'architettura.

2.2 Backbone per l'estrazione di feature

2.2.1 ResNet

ResNet (*Residual Network*), proposta da He et al. [1], introduce il concetto di *residual learning*, basato sull'utilizzo di connessioni residue (*skip connections*), con l'obiettivo di facilitare l'ottimizzazione di reti neurali molto profonde.

Un aspetto centrale affrontato in modo sistematico da ResNet riguarda il problema della degradazione delle prestazioni nelle reti profonde. Tale fenomeno non è riconducibile all'overfitting, il quale si manifesterebbe con un errore di training basso e un errore di test alto. Si osserva, invece, un aumento dell'errore anche sul training set: il problema non riguarda la generalizzazione, ma è legato a specifiche difficoltà di ottimizzazione delle *plain networks*¹: all'aumentare del numero di layer, diventa più difficile per l'algoritmo di ottimizzazione individuare parametri che permettano di approssimare correttamente la mappatura desiderata, anche quando tale mappatura è teoricamente rappresentabile dalla rete.

Le architetture residue come ResNet affrontano questo problema a livello architetturale: anziché richiedere che ogni blocco apprenda direttamente l'intera trasformazione $H(x)$, le reti di questo tipo riformulano l'obiettivo di ciascun blocco in termini di funzione residua $F(x)$. Questa riformulazione semplifica il compito di ottimizzazione e permette alla rete di sfruttare in maniera efficiente l'aumento di profondità, invece di subire la degradazione delle prestazioni.

¹ Reti prive di connessioni residue.

Più precisamente, la funzione residua $F(x)$ è definita come differenza tra mappatura desiderata e input del blocco.

Indicando con:

- x : input del blocco residuo;
- $H(x)$: mappatura desiderata, ovvero la trasformazione che il blocco dovrebbe apprendere;
- $F(x)$: funzione residua appresa dai layer del blocco;
- y : output del blocco residuo.

si definisce la funzione residua come:

$$F(x) = H(x) - x, \quad (2.2)$$

da cui segue:

$$H(x) = F(x) + x. \quad (2.3)$$

L'output del blocco può pertanto essere espresso come:

$$y = F(x) + x. \quad (2.4)$$

Il termine x viene trasferito direttamente all'uscita mediante una connessione che bypassa uno o più layer del blocco. Tale collegamento prende il nome di *skip connection* e costituisce l'elemento caratterizzante delle architetture ResNet.

La presenza della *skip connection* consente al blocco di apprendere soltanto la trasformazione necessaria rispetto all'input. Nel caso limite in cui la mappatura desiderata sia prossima alla funzione identità, il residuo $F(x)$ tende a essere prossimo a zero. L'output del blocco può quindi essere approssimato come:

$$y \approx 0 + x = x. \quad (2.5)$$

In questo modo, il blocco non è costretto ad apprendere una trasformazione completa quando questa non è necessaria, ma può concentrarsi sull'apprendimento delle modifiche da apportare all'input per ottenere la rappresentazione desiderata. Questo meccanismo

facilita la propagazione dell'informazione attraverso reti profonde e rende più agevole l'ottimizzazione dei parametri durante l'addestramento.

2.2.2 DenseNet

DenseNet, proposta da Huang et al. [2], riprende l'idea delle connessioni tra layer introdotta da ResNet, ma la sviluppa in modo più radicale: mentre ResNet collega l'input di ciascun blocco residuale al rispettivo output sommando l'identità dopo aver bypassato due o tre layer intermedi, in DenseNet ogni layer è connesso a *tutti* i layer successivi all'interno dello stesso blocco (denominato *dense block*).

Concretamente, l'input di ogni layer è dato dalla concatenazione delle feature map prodotte da tutti i layer precedenti nel blocco, anziché dalla sola somma usata da ResNet.

A titolo esemplificativo, si consideri un blocco formato da L layer con input iniziale x_0 . Il primo layer riceve x_0 e produce l'output x_1 . Il secondo layer riceve la concatenazione $[x_0, x_1]$ e produce x_2 . Il terzo layer riceve $[x_0, x_1, x_2]$, e così via: l' n -esimo layer riceve la concatenazione di tutti gli output precedenti $[x_0, x_1, \dots, x_{n-1}]$. Questo schema fa sì che il numero di connessioni cresca quadraticamente con la profondità del blocco (per un blocco di L layer si hanno $L(L+1)/2$ connessioni), favorendo il riutilizzo delle feature e mitigando il problema del vanishing gradient.

Formalmente, l'output dell' ℓ -esimo layer è dato da $x_\ell = H_\ell([x_0, x_1, \dots, x_{\ell-1}])$, dove $[x_0, x_1, \dots, x_{\ell-1}]$ indica la concatenazione delle feature map prodotte dai layer precedenti e H_ℓ è una funzione composta (tipicamente Batch Normalization, ReLU e convoluzione). Ogni layer produce un numero fisso di nuove feature map, detto *growth rate* (k), che determina quanto rapidamente cresce il numero di canali all'interno del blocco.

Per evitare che questa crescita diventi insostenibile, tra un dense block e il successivo vengono inseriti dei *transition layer*, che riducono sia la risoluzione (tramite pooling) sia il numero di canali (tramite una convoluzione 1×1).

2.2.3 EfficientNet

EfficientNet, proposta da Tan e Le [6], nasce con l'obiettivo di massimizzare l'Accuracy a parità di budget computazionale. A differenza degli approcci tradizionali, che scalano una sola dimensione della rete (ad esempio aumentando la risoluzione dell'input o semplicemente la profondità della rete), gli autori citati osservano che le tre dimensioni di scaling non sono indipendenti: bilanciarle tutte, contemporaneamente, produce risultati migliori rispetto a scalarne una sola.

Le tre dimensioni in questione sono:

- **Profondità**: numero di layer della rete;
- **Larghezza**: numero di canali per layer;
- **Risoluzione**: dimensione dell'immagine in input.

EfficientNet scala queste dimensioni tramite il *compound scaling method*: dato un coefficiente ϕ che rappresenta le risorse computazionali disponibili, profondità, larghezza e risoluzione vengono scalate secondo:

$$d = \alpha^\phi, \quad w = \beta^\phi, \quad r = \gamma^\phi, \quad \text{con} \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2, \quad (2.6)$$

dove α, β, γ sono costanti trovate tramite grid search su un modello base (*EfficientNet-B0*, ottenuto tramite *NAS*, Neural Architecture Search). Queste costanti stabiliscono come ripartire l'aumento di risorse tra le tre dimensioni; è invece il coefficiente ϕ , scelto dall'utente, a determinare quante risorse complessive si vogliono impiegare. Ogni incremento unitario di ϕ porta a un raddoppio approssimativo dei FLOPs² complessivi rispetto al modello base, con un fattore di crescita dell'ordine di 2^ϕ .

² *Floating point operations*, il numero di operazioni in virgola mobile richieste per un'inferenza.

2.3 Analisi comparativa

Le architetture presentate affrontano da diverse prospettive il problema di rendere le reti neurali sempre più profonde ed efficienti:

- **ResNet**: introduce l'apprendimento residuo tramite le *skip connection*, semplificando il problema di ottimizzazione delle reti profonde e permettendo di sfruttare in maniera efficace l'aumento di profondità della rete;
- **DenseNet**: sviluppa l'idea di connessioni tra layer, favorendo il riutilizzo delle feature tramite connessioni dense;
- **EfficientNet**: si concentra sull'ottimizzazione dell'equilibrio tra profondità, larghezza e risoluzione dell'immagine, per massimizzare l'Accuracy con un budget prefissato di potenza di calcolo.

Le prime due architetture agiscono sulla connettività interna della rete, mentre EfficientNet agisce sulla dimensione globale dell'architettura.

Una delle domande a cui questa tesi sperimentale tenta di rispondere è: come reagiscono questi modelli a diverse tipologie di perdita controllata di informazioni nel training set?

Capitolo 3

Pipeline sperimentale e metodologia

3.1 Dataset e task

Per realizzare la tesi sono stati utilizzati due dataset:

- *Oxford-IIIT Pet* [4], composto da immagini di cani e gatti suddivise in 37 classi. Il dataset comprende un file di annotazione che associa a ogni immagine una tupla (`image_path`, `micro_label`, `macro_label`), ove:
 - `micro_label` identifica la razza tra le 37 disponibili;
 - `macro_label` identifica la specie, ovvero cane o gatto.

Il dataset è diviso in tre sottoinsiemi: training (70%), validazione (15%) e test (15%). La suddivisione è effettuata in modo da preservare l'equa distribuzione delle 37 classi.

- *CIFAR-100* [3], composto da 100 classi raggruppate in 20 superclassi. Ogni super-classe comprende 5 classi, per un totale di 100 *fine label*. Per contenere il costo computazionale degli esperimenti, è stato estratto inizialmente un sottoinsieme di 50 campioni per ciascuna classe. Successivamente tale numero è stato aumentato a 200 campioni per classe, poiché la quantità iniziale produceva un risultato caratterizzato da variabilità e rumore.

Il campionamento è stato effettuato prima della suddivisione stratificata del dataset, secondo la proporzione 70%/15%/15% tra training, validazione e test.

3.2 Architettura multi-task

Delle famiglie architetturali presentate nella Sezione 2.2, sono state selezionate quattro varianti come backbone per gli esperimenti: **ResNet18**, **ResNet50**, **DenseNet121** ed **EfficientNet-B0**.

L'integrazione dell'apprendimento multi-task su tali backbone segue uno schema standard, noto come *hard parameter sharing*: un unico backbone convoluzionale condiviso è utilizzato da due head di classificazione indipendenti.

Il backbone elabora l'immagine fino al GAP¹, producendo un vettore di embedding la cui dimensione dipende dall'architettura:

- 512 per ResNet18;
- 2048 per ResNet50;
- 1024 per DenseNet121;
- 1280 per EfficientNet-B0.

Questo vettore è utilizzato in parallelo da entrambe le head.

Le due head sono costituite da un singolo layer lineare (`Linear(feature_dim, num_classes)`). Le proiezioni delle due head sono, rispettivamente, su 37 classi (razza) e 2 classi (specie) per Oxford-IIIT Pet; su 100 classi (*fine*) e 20 classi (*coarse*) per CIFAR-100.

La funzione di loss è una combinazione pesata delle due cross-entropy, tramite i coefficienti λ_{micro} e λ_{macro} :

$$\mathcal{L}_{\text{tot}} = \lambda_{\text{micro}} \cdot \mathcal{L}_{\text{micro}} + \lambda_{\text{macro}} \cdot \mathcal{L}_{\text{macro}}, \quad (3.1)$$

con $\lambda_{\text{micro}} = 1.0$ e $\lambda_{\text{macro}} = 0.5$: il task *fine* (micro) pesa dunque il doppio rispetto al task *coarse* (macro) nella loss totale.

3.3 Procedura di training dei modelli

3.3.1 Regime di addestramento

Tutti i backbone sono inizializzati con i pesi pre-addestrati su ImageNet, secondo l'approccio del *transfer learning* [7]. Come mostrato dagli stessi autori, la specializzazione dei layer più profondi al task originario può limitare la trasferibilità delle feature se

¹ GAP: *Global Average Pooling*. Operazione applicata al termine del backbone che riduce ciascuna mappa di feature a un singolo valore medio, trasformando così il volume di feature tridimensionale in un vettore monodimensionale utilizzabile dalle head di classificazione.

queste non vengono adattate al nuovo dominio; per questo motivo l'addestramento è condotto in modalità *full fine-tuning*: tutti i parametri del backbone rimangono aggiornabili, insieme a quelli delle due head, anziché congelare il backbone e addestrare solo le head (*feature extraction*).

Il backbone adatta così i propri filtri alle caratteristiche peculiari del dataset, invece di limitarsi alle feature generiche di ImageNet.

3.3.2 Ottimizzazione

I pesi sono aggiornati tramite l'ottimizzatore *Adam*, con learning rate fissato a 0.001, minimizzando la loss combinata $\mathcal{L}_{\text{tot}}$ definita nella Sezione 3.2. L'addestramento è condotto per un massimo di 50 epoche, con dimensione del batch pari a 32.

3.3.3 Trasformazioni sui dati

Tutte le immagini sono ridimensionate a 224×224 pixel e normalizzate secondo media e deviazione standard di ImageNet ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$), ovvero gli stessi valori usati durante il pre-training su ImageNet.

Nel set di training, sull'immagine ridimensionata è applicato un *random horizontal flip* con probabilità $p = 0.5$, come unica forma di data augmentation. Nei set di validazione e test non è applicata alcuna trasformazione stocastica: le immagini sono unicamente ridimensionate e normalizzate, per garantire una valutazione deterministica e riproducibile.

Tutte le operazioni stocastiche (suddivisione del dataset, degradazione controllata, augmentation) sono governate da un unico seed fisso che garantisce il determinismo² dell'intera pipeline.

3.3.4 Early stopping

Il training è monitorato a ogni epoca, valutando la loss totale $\mathcal{L}_{\text{tot}}$ (calcolata con gli stessi coefficienti λ_{micro} e λ_{macro} definiti in Sezione 3.2) sul set di validazione. È adottato un criterio di *early stopping* con pazienza pari a 6 epoche: se la loss di validazione non migliora per 6 epoche consecutive, l'addestramento viene interrotto anticipatamente, ripristinando i pesi del modello relativi all'epoca in cui si è registrata la loss di validazione minima, corrispondente al miglior modello.

² L'esperimento si può riprodurre molteplici volte ottenendo lo stesso risultato, grazie al seed fissato come parametro.

3.4 Degradazione controllata del training set

Per studiare la robustezza delle architetture a diverse forme di perdita di informazione, è stata implementata una procedura di degradazione controllata del training set, articolata in due meccanismi indipendenti: *macro-drop* e *micro-drop*. Entrambi sono applicati esclusivamente al training set, lasciando i set di validazione e test sempre integri. Le percentuali di degradazione applicate vanno dallo 0% al 40%, con incrementi del 5%, e l'intera procedura è determinata da un seed.

Macro-drop

Il *macro-drop* simula una condizione di scarsità di dati, riducendo il numero di campioni disponibili senza mai eliminare interamente le classi. La rimozione è stratificata per classe: per ogni classe viene rimossa la medesima quantità proporzionale di campioni; eventuali resti derivanti da arrotondamento vengono distribuiti in modo che alcune classi perdano un campione in più o in meno rispetto alle altre.

Tutte le classi restano quindi rappresentate nel training set, con un numero di esempi ridotto.

Micro-drop

Il *micro-drop* simula la scomparsa di intere classi durante il training. A ogni percentuale di degradazione, un sottoinsieme di classi (razze per Oxford-IIIT Pet, classi *fine* per CIFAR-100) viene selezionato ed eliminato dal training set, il quale corrisponde al 70% del dataset totale.

Per Oxford-IIIT Pet, la selezione è ristretta alle sole razze canine (circa 3484 immagini nel training set, contro le 1659 delle razze feline), mentre per CIFAR-100 non è necessaria alcuna restrizione: le classi da eliminare vengono scelte casualmente, tramite seed, tra tutte le 100 classi *fine*. La scelta di filtrare Oxford-IIIT Pet nasce dallo sbilanciamento del dataset tra le due specie (25 razze canine contro 12 feline): una selezione casuale su tutte le 37 razze avrebbe potuto, in alcuni casi sfortunati, colpire in prevalenza o esclusivamente le razze feline, già numericamente sottorappresentate, alterando in modo incontrollato l'equilibrio tra le due categorie di livello superiore (specie).

Il numero di classi da eliminare è determinato a partire dalla percentuale di drop richiesta, calcolata sul numero di immagini anziché sul numero di classi. Ad esempio, un drop del 20% su 3484 immagini corrisponde a circa 696 immagini da rimuovere; poiché ogni

razza dispone di circa 140 immagini nel training set, questo equivale a circa 4.97 classi. L'algoritmo elimina le classi in ordine progressivo, secondo il seed fisso, rimuovendo per intero ciascuna classe prima di passare alla successiva: nell'esempio, le prime 4 classi vengono eliminate completamente, mentre la quinta viene svuotata solo parzialmente, fino al raggiungimento del numero target di immagini.

Ad esempio, con *micro-drop* al 20% su Oxford-IIIT Pet, applicato con seed 777, sono state selezionate e rimosse le razze *Yorkshire Terrier*, *English Setter*, *German Shorthaired Pointer* e *Boxer*.

La head relativa al task coarse (macro, ovvero specie o superclasse) continua ad addestrarsi regolarmente sui campioni delle classi superstiti: poiché la selezione delle istanze da escludere opera su singole categorie di livello fine (micro), le restanti classi appartenenti alla medesima superclasse continuano a fornire esempi per l'aggiornamento della head coarse.

Effetto sulla valutazione

Indipendentemente dal tipo e dall'entità della degradazione applicata in training, la valutazione finale è sempre condotta sull'insieme completo delle classi (37 per Oxford-IIIT Pet, 100 per CIFAR-100). Con il *micro-drop*, questo significa che il modello viene valutato anche su classi che non ha mai osservato durante l'addestramento. Tale condizione ha conseguenze dirette sulle metriche di valutazione, discusse nella Sezione 3.5.

3.5 Metriche di valutazione

I risultati degli esperimenti sono raccolti in tre tipologie di file CSV, con scopi complementari.

Log di training

Per ogni modello viene generato un unico file di log che aggrega lo storico di tutte le run effettuate su quel backbone; nel caso di ResNet18, ad esempio, il file è denominato `resnet18_training_log.csv`. Ogni riga corrisponde a un'epoca di una specifica run, identificata dal tipo di degradazione, dalla percentuale e dal seed, e riporta le corrispondenti loss di training e di validazione.

Report globale

Il report globale presenta, per ogni combinazione di tipologia di degradazione, percentuale e seed, tre valori aggregati calcolati sul test set completo (37 classi per Oxford-IIIT Pet, 100 per CIFAR-100):

- **Accuracy**: rapporto tra predizioni corrette e totale delle predizioni, calcolata sul task micro;
- **Macro F1 globale**: media aritmetica dell'F1-score calcolato su tutte le classi del task micro, incluse quelle non presenti nel training della run corrente;
- **Macro F1 note**: media aritmetica dell'F1-score calcolato esclusivamente sulle classi effettivamente presenti nel training della run corrente. L'insieme delle classi note viene ricostruito a partire dal seed della run, ripetendo la stessa procedura di degradazione applicata in fase di addestramento.

La distinzione tra le due varianti di F1 è motivata dall'effetto del *micro-drop* sulle metriche: le classi escluse dal training ricevono soltanto gradienti che abbassano il logit della loro unità di output corrispondente e, in fase di test, ottengono tipicamente F1 pari a zero (nessuna predizione corretta per quella classe).

Includendo queste classi nel calcolo della Macro F1 globale, la metrica risulta penalizzata dalle classi escluse, indipendentemente dalla reale capacità del modello sulle classi effettivamente apprese.

La *Macro F1 note* isola questo effetto, restituendo una misura della performance del modello limitata a ciò che ha potuto effettivamente imparare.

Report per classe

Il terzo file riporta, per ogni combinazione di tipologia di degradazione, percentuale e seed, una riga per ciascuna delle classi del task micro, con le seguenti informazioni:

- l'indicazione se la classe sia stata osservata o meno durante il training della run corrente;
- precision, recall, F1-score e support, calcolati sul test set.

Questo livello di dettaglio consente di analizzare l'impatto della degradazione classe per classe, distinguendo il comportamento del modello sulle classi note da quello sulle classi mai osservate in training.

Capitolo 4

Risultati e discussione

Nelle tabelle di questo capitolo le metriche sono riportate come valori compresi tra 0 e 1; le differenze discusse nel testo sono espresse in punti percentuali (ad esempio, una differenza di 0.05 corrisponde a 5 punti).

4.1 Risultati sul dataset Oxford-IIIT Pet

Analisi Macro-drop

Come mostrato in Figura 4.1, il macro-drop non produce un trend di degradazione riconoscibile: per tutte le architetture le curve oscillano entro una banda ristretta al variare della percentuale di rimozione, senza una direzione definita.

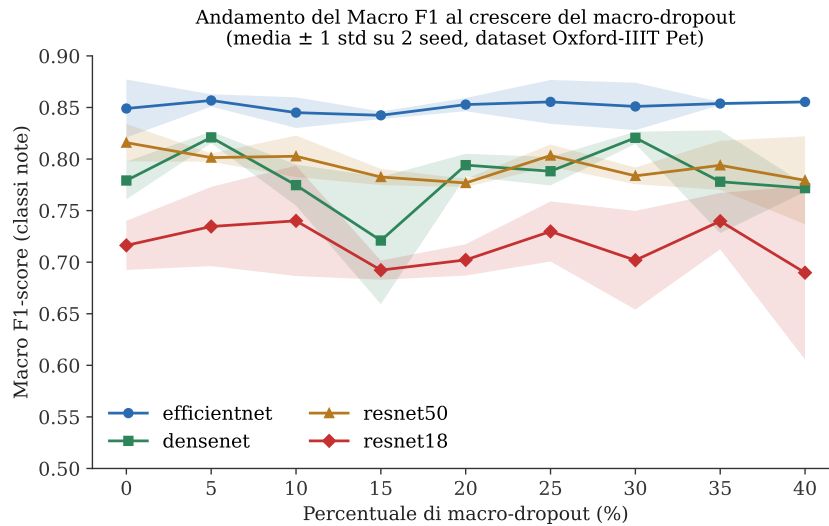


Figura 4.1: Andamento del Macro F1-score al variare della percentuale di macro-drop, per le quattro architetture testate sul dataset Oxford-IIIT Pet (in tale scenario tutte le classi del dataset rimangono rappresentate nel training set). Le bande rappresentano ± 1 deviazione standard calcolata su due seed (42, 777).

Tabella 4.1: Accuracy, Precision e F1-score al variare della percentuale di macro-drop, per le quattro architetture sul dataset Oxford-IIIT Pet. Valori riportati come media $\pm$ deviazione standard su due seed (42, 777).

Modello	Drop %	Accuracy	Precision	F1-score
EfficientNet	0	0.851 ± 0.029	0.862 ± 0.021	0.849 ± 0.028
	20	0.853 ± 0.009	0.868 ± 0.001	0.853 ± 0.006
	40	0.855 ± 0.002	0.864 ± 0.000	0.855 ± 0.001
DenseNet	0	0.780 ± 0.017	0.806 ± 0.010	0.779 ± 0.018
	20	0.797 ± 0.011	0.820 ± 0.007	0.794 ± 0.011
	40	0.772 ± 0.004	0.799 ± 0.004	0.772 ± 0.003
ResNet50	0	0.818 ± 0.017	0.836 ± 0.009	0.816 ± 0.018
	20	0.777 ± 0.006	0.805 ± 0.002	0.777 ± 0.004
	40	0.781 ± 0.038	0.810 ± 0.027	0.779 ± 0.043
ResNet18	0	0.719 ± 0.022	0.763 ± 0.011	0.716 ± 0.024
	20	0.704 ± 0.015	0.758 ± 0.011	0.702 ± 0.015
	40	0.683 ± 0.087	0.745 ± 0.051	0.690 ± 0.084

I valori riportati in Tabella 4.1 confermano quanto osservato graficamente: Accuracy, Precision e F1-score restano stabili tra lo 0% e il 40% di macro-drop per ciascuna architettura, con variazioni contenute di pochi punti percentuali. Questo risultato è atteso, poiché la riduzione uniforme degli esempi in ogni classe mantiene invariata la distribuzione relativa del dataset, lasciando pressoché inalterato il problema di classificazione anche con un numero minore di esempi di training.

Un'eccezione riguarda ResNet18 al 40% di drop, dove la deviazione standard tra i due seed risulta sensibilmente più elevata rispetto agli altri casi (± 0.087 in Accuracy), segnale di una maggiore sensibilità di questa architettura al rumore di ottimizzazione e all'inizializzazione casuale dei parametri delle head lineari, anche a fronte di una riduzione uniforme dei campioni.

Analisi Micro-drop

Come mostrato in Figura 4.2 e in Tabella 4.2, il micro-drop produce un trend di degradazione marcato: il Macro F1-score¹ diminuisce al crescere della percentuale di rimozione delle classi, con intensità diverse tra le quattro architetture testate sul dataset Oxford-IIIT Pet.

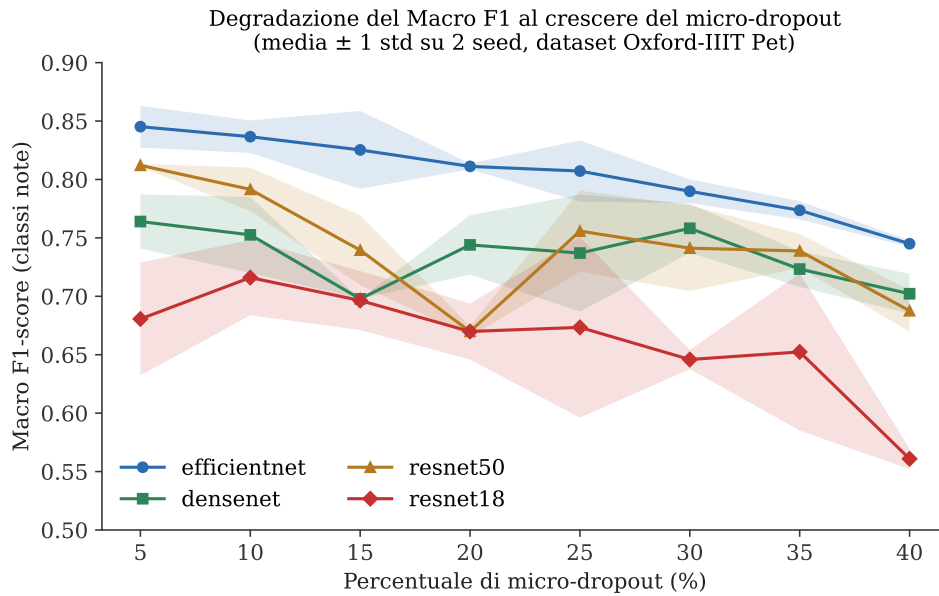


Figura 4.2: Andamento del Macro F1-score sulle classi note al variare della percentuale di micro-drop, per le quattro architetture testate sul dataset Oxford-IIIT Pet. Le bande rappresentano ± 1 deviazione standard calcolata su due seed (42, 777).

¹ Media semplice dell'F1-score calcolato per ogni classe, limitatamente alle classi presenti nel training set (escluse quindi le razze completamente rimosse dal micro-drop).

Tabella 4.2: Accuracy (globale sull'intero test set), Precision (macro, classi note) e F1-score (macro, classi note) al variare della percentuale di micro-drop, per le quattro architetture sul dataset Oxford-IIIT Pet. Valori riportati come media $\pm$ deviazione standard su due seed (42, 777).

Modello	Drop %	Accuracy	Precision	F1-score
EfficientNet	0	0.851 ± 0.029	0.862 ± 0.021	0.849 ± 0.028
	20	0.753 ± 0.014	0.798 ± 0.009	0.811 ± 0.002
	40	0.633 ± 0.016	0.699 ± 0.005	0.745 ± 0.003
DenseNet	0	0.780 ± 0.017	0.806 ± 0.010	0.779 ± 0.018
	20	0.691 ± 0.015	0.750 ± 0.007	0.744 ± 0.025
	40	0.592 ± 0.023	0.672 ± 0.016	0.702 ± 0.017
ResNet50	0	0.818 ± 0.017	0.836 ± 0.009	0.816 ± 0.018
	20	0.628 ± 0.012	0.708 ± 0.025	0.670 ± 0.004
	40	0.578 ± 0.022	0.684 ± 0.007	0.688 ± 0.018
ResNet18	0	0.719 ± 0.022	0.763 ± 0.011	0.716 ± 0.024
	20	0.628 ± 0.008	0.673 ± 0.010	0.670 ± 0.024
	40	0.479 ± 0.028	0.585 ± 0.007	0.561 ± 0.009

I risultati evidenziano un trend di degradazione del Macro F1-score al crescere della percentuale di micro-drop per tutte le architetture, sebbene con intensità differenti. Si nota inoltre come l'Accuracy subisca una flessione più marcata rispetto all'F1-score delle classi note: l'Accuracy è valutata sull'intero test set e risente direttamente degli errori sulle classi rimosse. Anche l'F1-score delle classi note, tuttavia, non è del tutto indipendente da tali classi: i campioni di test appartenenti alle classi rimosse vengono necessariamente assegnati a una classe nota, comparando come falsi positivi e riducendone la Precision. Parte del calo dell'F1-score delle classi note è quindi riconducibile a questo effetto, e non soltanto a una minore capacità del modello sulle classi apprese.

EfficientNet ottiene le prestazioni più elevate a ogni livello di drop testato e, ai livelli di drop del 20% e del 40%, è anche il modello con la minore variabilità dell'F1-score tra i due seed. DenseNet, pur partendo da valori inferiori, mostra la degradazione relativa più contenuta tra il 5% e il 40% di drop, risultando il modello più resiliente in termini relativi anche se non il migliore in assoluto.

ResNet18 rappresenta il caso più critico: presenta il Macro F1-score più basso in quasi tutte le configurazioni ed è anche il modello con la maggiore variabilità tra i seed, evidenziando una sensibilità non trascurabile sia alla perdita di classi sia all'inizializzazione casuale. ResNet50 mostra invece un andamento meno regolare, con oscillazioni non monotone lungo l'intervallo di drop testato, verosimilmente dovute a una particolare combinazione di classi rimosse.

Un aspetto metodologicamente rilevante emerge dal confronto tra i due seed: l'ampiezza delle bande di errore in Figura 4.2 varia sensibilmente tra le architetture, indicando che una valutazione basata su un singolo seed avrebbe potuto portare a conclusioni fuorvianti.

4.2 Risultati sul dataset CIFAR-100

Per il dataset CIFAR-100 sono state addestrate e valutate due sole architetture, EfficientNet e ResNet50, a causa di vincoli di tempo e di risorse di calcolo disponibili. I risultati riportati in questa sezione fanno riferimento a una singola run (seed 42).

Analisi Macro-drop

Come mostrato in Figura 4.3, anche su CIFAR-100 il macro-drop non produce un trend di degradazione riconoscibile: le curve di entrambe le architetture restano pressoché piatte lungo l'intero intervallo di percentuali testate.

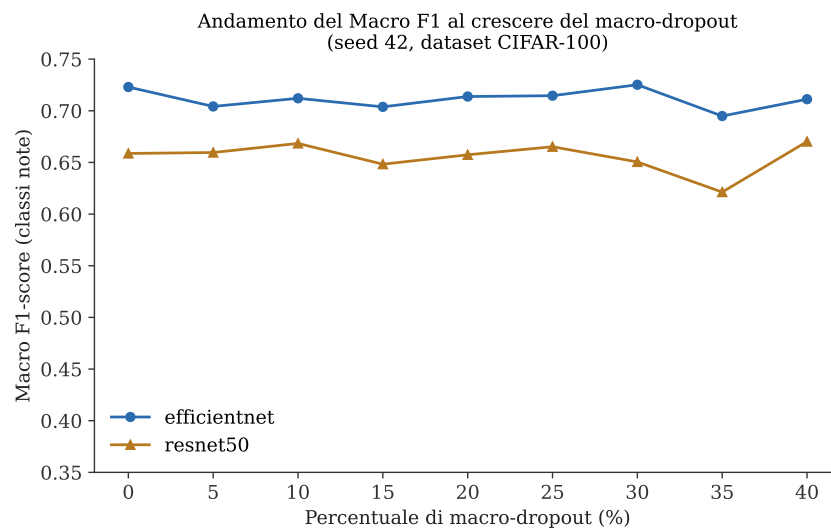


Figura 4.3: Andamento del Macro F1-score al variare della percentuale di macro-drop, per EfficientNet e ResNet50 sul dataset CIFAR-100 (seed 42).

Tabella 4.3: Accuracy, Precision e F1-score al variare della percentuale di macro-drop, per EfficientNet e ResNet50 sul dataset CIFAR-100 (seed 42).

Modello	Drop %	Accuracy	Precision	F1-score
EfficientNet	0	0.723	0.738	0.723
	20	0.714	0.728	0.714
	40	0.712	0.725	0.711
ResNet50	0	0.659	0.689	0.659
	20	0.655	0.680	0.657
	40	0.672	0.694	0.670

I valori in Tabella 4.3 confermano il comportamento osservato graficamente, con variazioni minime tra lo 0% e il 40% di macro-drop per entrambe le architetture. Il risultato è

coerente con quanto osservato sul dataset Oxford-IIIT Pet, ed è atteso per lo stesso motivo: la riduzione uniforme degli esempi per ciascuna classe non altera la difficoltà relativa del problema di classificazione.

Analisi Micro-drop

Come mostrato in Figura 4.4 e in Tabella 4.4, anche su CIFAR-100 il micro-drop produce un trend di degradazione chiaro e monotono del Macro F1-score al crescere della percentuale di classi rimosse dal training, in linea con quanto osservato sul dataset Oxford-IIIT Pet.

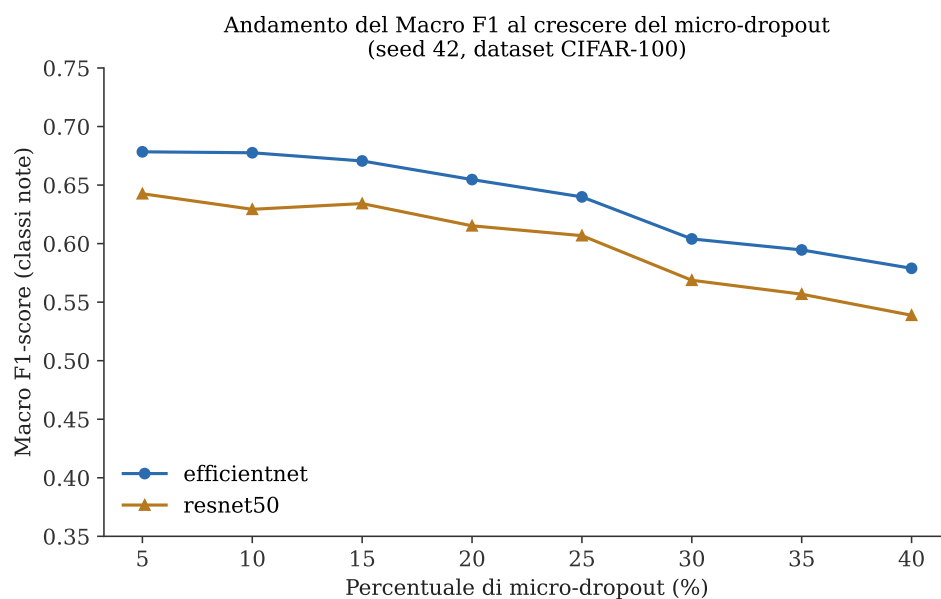


Figura 4.4: Andamento del Macro F1-score sulle classi note al variare della percentuale di micro-drop, per EfficientNet e ResNet50 sul dataset CIFAR-100 (seed 42).

Tabella 4.4: Accuracy (globale sull'intero test set), Precision (macro, classi note) e F1-score (macro, classi note) al variare della percentuale di micro-drop, per EfficientNet e ResNet50 sul dataset CIFAR-100 (seed 42).

Modello	Drop %	Accuracy	Precision	F1-score
EfficientNet	0	0.723	0.738	0.723
	20	0.579	0.620	0.655
	40	0.452	0.489	0.579
ResNet50	0	0.659	0.689	0.659
	20	0.542	0.595	0.615
	40	0.412	0.469	0.539

Il confronto con i risultati ottenuti su Oxford-IIIT Pet (Sezione 4.1) evidenzia un comportamento coerente tra i due dataset: in entrambi i casi il micro-drop causa una degradazione progressiva del Macro F1-score al crescere della percentuale di classi rimosse, sebbene con intensità diverse a seconda dell'architettura e del dataset considerati. Su CIFAR-100 il calo risulta particolarmente marcato oltre il 25% di drop, con una perdita di circa 25 punti percentuali di Accuracy (fino a 27.1 per EfficientNet e 24.7 per ResNet50) tra il caso baseline e il 40% di rimozione. Questo risultato verrà approfondito nella discussione (Sezione 4.3).

4.3 Discussione sui risultati ottenuti

Il confronto tra i due dataset conferma alcuni pattern comuni e ne evidenzia altri maggiormente specifici di ciascun dataset.

Macro-drop. Su entrambi i dataset la riduzione uniforme degli esempi per classe non produce un trend di degradazione riconoscibile: le oscillazioni osservate restano contenute (con un intervallo di variazione del Macro F1 compreso tra 0.028 e 0.080 per Pet e tra 0.030 e 0.049 per CIFAR-100), confermando che questo tipo di degradazione non altera sostanzialmente la difficoltà relativa del problema.

Micro-drop. Come mostrato in Figura 4.5, la rimozione di intere classi produce una degradazione progressiva del Macro F1-score su entrambi i dataset, con un calo relativo compreso approssimativamente tra il 10% e il 22% dalla baseline al 40% di drop, a seconda dell'architettura considerata. La differenza principale tra i due dataset risiede nel punto di partenza: CIFAR-100 presenta prestazioni di baseline significativamente inferiori a quelle di Pet ($F1 \approx 0.68$ contro $F1 \approx 0.85$), verosimilmente a causa della maggiore complessità del task (100 classi contro 37 e risoluzione nativa delle immagini di soli 32×32 pixel che, pur interpolata a 224×224 , offre un livello di dettaglio visivo intrinsecamente inferiore rispetto a Oxford-IIIT Pet). Questo comporta che, a parità di percentuale di drop, le prestazioni assolute su CIFAR-100 risultino inferiori a quelle registrate su Pet. Il confronto a parità di percentuale va comunque interpretato con cautela: su Oxford-IIIT Pet la percentuale di micro-drop è calcolata sulle sole immagini delle razze canine (Sezione 3.4), per cui il 40% corrisponde a circa il 27% del training set e a 9 razze su 37, mentre su CIFAR-100 corrisponde a 40 classi su 100.

Si osserva inoltre che l'andamento della curva di degradazione è più irregolare su Pet, specialmente per ResNet50, mentre risulta più regolare e monotono su CIFAR-100. Questo potrebbe essere legato al numero maggiore di classi coinvolte in CIFAR-100, che rende l'effetto della rimozione di singole classi meno sensibile alla specifica combinazione di classi escluse a ciascuna soglia.

In entrambi i casi, il ranking tra le architetture si mantiene coerente: EfficientNet ottiene prestazioni superiori a ResNet50 a ogni livello di drop testato, su entrambi i dataset. Si tratta tuttavia di un vantaggio in termini di prestazioni assolute, non di robustezza: in termini di calo relativo, su CIFAR-100 ResNet50 perde meno di EfficientNet (-18.2% contro -19.9%), mentre su Pet il calo più contenuto è quello di DenseNet.

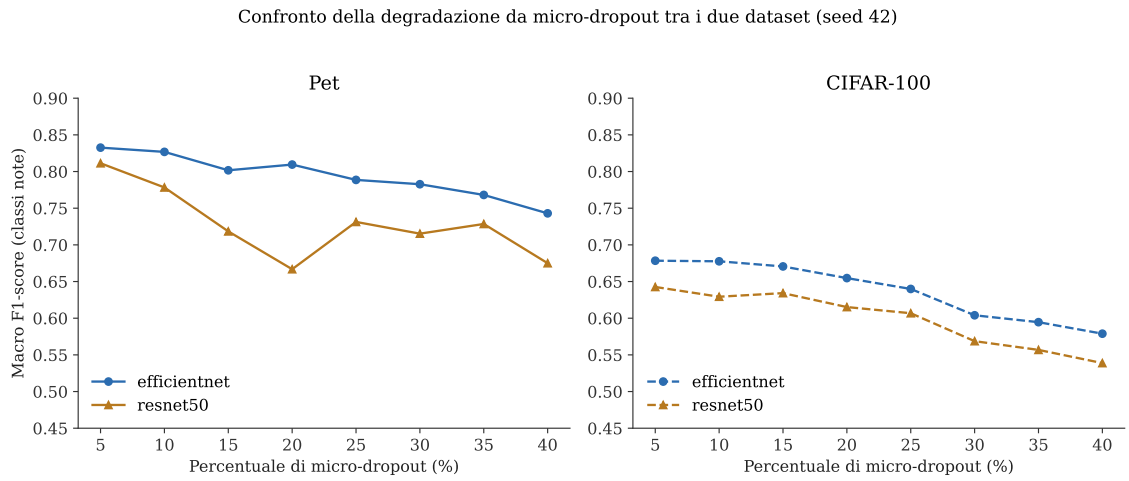


Figura 4.5: Confronto diretto dell'andamento del Macro F1-score al crescere della percentuale di micro-drop tra i due dataset (seed 42).

Capitolo 5

Approccio basato su reti prototipiche

5.1 Motivazione: il problema delle classi non viste

I risultati presentati nel Capitolo 4 hanno mostrato come il micro-drop, ossia la rimozione di intere classi dal training set, produca una degradazione significativa delle prestazioni di classificazione, a differenza del macro-drop, il cui impatto è risultato trascurabile su entrambi i dataset considerati. Un classificatore standard, tuttavia, non è in grado di riconoscere classi mai osservate durante l'addestramento: la soluzione più diretta con gli approcci tradizionali sarebbe un riaddestramento completo ogni volta che emerge una nuova classe da riconoscere, con un costo computazionale spesso proibitivo.

Il *few-shot learning* affronta questo problema proponendo modelli in grado di generalizzare a nuove classi a partire da un numero molto ridotto di esempi, senza richiedere un nuovo addestramento della rete. In questo capitolo viene adottato l'approccio delle reti prototipiche, che si presta naturalmente a questo scenario grazie alla sua semplicità concettuale e all'assenza di parametri aggiuntivi da apprendere per integrare nuove classi.

5.2 Reti prototipiche: principio di funzionamento

Le reti prototipiche, introdotte da Snell et al. [5], proiettano le immagini in uno spazio di embedding $f_\phi : \mathbb{R}^D \rightarrow \mathbb{R}^M$ in cui gli esempi di ciascuna classe k si concentrano attorno a un punto rappresentativo, detto *prototipo*, calcolato come media degli embedding del relativo support set S_k :

$$c_k = \frac{1}{|S_k|} \sum_{x \in S_k} f_\phi(x). \quad (5.1)$$

La classificazione di un nuovo esempio x avviene calcolando la distanza (euclidea al quadrato, nella formulazione originale) tra il suo embedding $f_\phi(x)$ e ciascun prototipo c_k , assegnando la classe corrispondente secondo la distribuzione softmax sulle distanze:

$$p_\phi(y = k \mid x) = \frac{\exp(-\|f_\phi(x) - c_k\|^2)}{\sum_{k'} \exp(-\|f_\phi(x) - c_{k'}\|^2)}. \quad (5.2)$$

Questo approccio rende semplice il funzionamento few-shot: è sufficiente disporre di un piccolo numero di esempi (*shot*) per costruire il prototipo di una nuova classe, senza la necessità di riaddestrare l'intero modello.

A differenza della formulazione originale, in cui l'estrattore f_ϕ viene addestrato in modo episodico, in questo lavoro i prototipi sono costruiti sulle feature di una rete addestrata con cross-entropy e mantenuta congelata: il modello adottato è quindi un classificatore a prototipi ispirato alle reti prototipiche.

5.3 Setup e campionamento

La valutazione del modello prototipico è stata condotta secondo lo scenario *Generalized Few-Shot*: per ciascuna configurazione di micro-drop del Capitolo 4, i prototipi vengono costruiti sia per le classi note (dall'intero training set) sia per le classi escluse (a partire da 5 immagini di supporto per classe), e la classificazione avviene sul test set completo, il quale comprende entrambe le categorie di classi. Questo scenario è stato preferito a una valutazione few-shot ristretta alle sole classi non viste poiché si collega direttamente al problema individuato nel Capitolo 4: verificare se il recupero delle classi mancanti tramite pochi esempi comprometta le prestazioni sulle classi già apprese.

Come estrattore di feature è stato riutilizzato, mantenendone i pesi congelati¹, il modello già addestrato nel Capitolo 4 per ciascuna architettura e percentuale di micro-drop, evitando così un nuovo addestramento e mantenendo la coerenza tra i due esperimenti.

Per il dataset Oxford-IIIT Pet la valutazione è stata ripetuta 30 volte per ciascuna configurazione, con campionamento casuale del support set a ogni episodio, riportando media e deviazione standard sulle metriche. Per CIFAR-100, a causa di vincoli di tempo e di risorse di calcolo, la valutazione è stata condotta su un singolo episodio; tale limite è stato tenuto in considerazione nell'interpretazione dei risultati riportati in Sezione 5.4. Per entrambi i dataset il campionamento del support set è governato dal seed 777; per CIFAR-100 il modello riutilizzato è quello addestrato con seed 42, l'unico disponibile (Sezione 4.2).

5.4 Risultati ottenuti e discussione

Come mostrato in Figura 5.1 e in Tabella 5.1, su Oxford-IIIT Pet EfficientNet mantiene le prestazioni più elevate sulle classi note e, in media sui tre livelli di drop, anche su

¹ I pesi del modello preso in considerazione non vengono aggiornati. La rete è impiegata per il calcolo degli embedding delle immagini, successivamente utilizzati per costruire i prototipi delle classi.

quelle recuperate tramite few-shot, confermando il pattern di robustezza già osservato nel Capitolo 4. Le classi recuperate mostrano un'Accuracy sistematicamente più bassa rispetto a quelle note (tra circa 20 e 33 punti percentuali di differenza) e una variabilità marcatamente maggiore tra gli episodi, coerente con l'aspettativa che 5 soli esempi di supporto per classe producano prototipi meno stabili rispetto a quelli costruiti sull'intero training set.

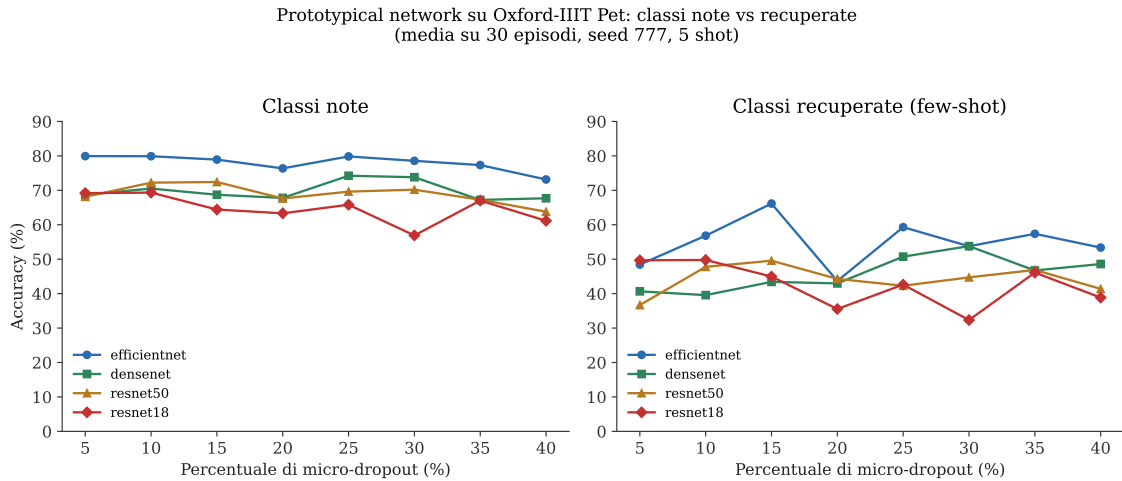


Figura 5.1: Accuracy del modello prototipico su Oxford-IIIT Pet, separata tra classi note (sinistra) e classi recuperate tramite few-shot (destra), al variare della percentuale di micro-drop. Media su 30 episodi, seed 777, 5 shot per classe.

Tabella 5.1: Accuracy (%) sulle classi note e sulle classi recuperate tramite few-shot, per le quattro architetture sul dataset Oxford-IIIT Pet. Media $\pm$ deviazione standard su 30 episodi.

Modello	Drop %	Acc. Note	Acc. Ignote	Classi escluse
EfficientNet	5	79.93 $\pm$ 1.27	48.44 $\pm$ 6.43	1
	20	76.37 $\pm$ 1.09	43.67 $\pm$ 4.43	4
	40	73.15 $\pm$ 1.45	53.37 $\pm$ 4.08	9
DenseNet	5	68.66 $\pm$ 1.35	40.67 $\pm$ 8.32	1
	20	67.79 $\pm$ 1.37	42.97 $\pm$ 5.40	4
	40	67.69 $\pm$ 1.26	48.60 $\pm$ 3.24	9
ResNet50	5	68.11 $\pm$ 1.55	36.67 $\pm$ 10.78	1
	20	67.63 $\pm$ 1.33	44.28 $\pm$ 5.12	4
	40	63.78 $\pm$ 1.59	41.38 $\pm$ 2.60	9
ResNet18	5	69.18 $\pm$ 1.14	49.67 $\pm$ 9.12	1
	20	63.32 $\pm$ 1.22	35.53 $\pm$ 4.21	4
	40	61.15 $\pm$ 1.57	38.85 $\pm$ 4.40	9

Confrontando questi risultati con la baseline del Capitolo 4 (senza recupero few-shot), l'introduzione del modello prototipico comporta generalmente un peggioramento modera-

to delle prestazioni sulle classi note per tutte e quattro le architetture, con perdite comprese tra circa 1 e 13 punti percentuali a seconda del modello e della percentuale di drop. Questo suggerisce che l'aggiunta dei prototipi delle classi non viste introduce un certo grado di interferenza nello spazio degli embedding condiviso, più marcata per ResNet50 e minore per EfficientNet.

I risultati su CIFAR-100 (Tabella 5.2), pur limitati a un singolo episodio, mostrano un divario più contenuto tra classi note e classi recuperate rispetto a quanto rilevato su Oxford-IIIT Pet. Sebbene tale confronto debba tenere conto della diversa metrica impiegata nei report (Macro F1-score per CIFAR-100 rispetto all'Accuracy per Oxford-IIIT Pet), una possibile spiegazione è che la minore separazione tra i prototipi su CIFAR-100 (100 classi totali contro 37) renda la distanza mediamente inferiore e quindi lo spazio decisionale più uniforme tra classi note e classi recuperate.

Tabella 5.2: Macro F1-score (%) sulle classi note e sulle classi recuperate tramite few-shot, per EfficientNet e ResNet50 sul dataset CIFAR-100 (singolo episodio, seed 777).

Modello	Drop %	F1 Note	F1 Ignote
EfficientNet	5	68.35	66.84
	20	65.17	61.80
	40	59.60	51.04
ResNet50	5	60.32	57.33
	20	57.04	50.65
	40	54.32	42.82

In sintesi, il modello prototipico si conferma una soluzione efficace per recuperare parzialmente la capacità di riconoscere classi escluse dal training, sebbene con prestazioni inferiori rispetto a quelle ottenute sulle classi apprese in modo completo e con un costo moderato sulle prestazioni delle classi note, più evidente su Pet che su CIFAR-100.

Capitolo 6

Conclusioni e sviluppi futuri

6.1 Sintesi e limiti dello studio

Il presente elaborato ha analizzato l'impatto di due forme di degradazione controllata del dataset di training sulle prestazioni di classificazione di diverse architetture convoluzionali. I dataset scelti sono Oxford-IIIT Pet e CIFAR-100, mentre le due tipologie di degradazione sono il macro-drop, che riduce uniformemente gli esempi di ciascuna classe, e il micro-drop, che rimuove del tutto alcune classi.

I risultati hanno mostrato con costanza come il macro-drop non produca una degradazione significativa delle prestazioni, mentre il micro-drop comprometta progressivamente il Macro F1-score al crescere della percentuale di classi rimosse, con un impatto relativo simile tra i due dataset ma livelli assoluti più critici su CIFAR-100, verosimilmente a causa della maggiore complessità intrinseca del task. In entrambi i casi, EfficientNet si è confermata l'architettura con le prestazioni assolute più elevate, mentre ResNet18, addestrata soltanto su Oxford-IIIT Pet, ha mostrato le maggiori criticità sia in termini di prestazioni assolute sia di stabilità tra le run.

A partire da questi risultati, è stato adottato un approccio a reti prototipiche per recuperare la capacità di riconoscere le classi escluse dal training tramite un numero ridotto di esempi di supporto. I risultati ottenuti su Oxford-IIIT Pet, valutati su 30 episodi indipendenti, hanno mostrato un recupero parziale ma apprezzabile delle classi non viste, a fronte di un peggioramento moderato delle prestazioni sulle classi note, più marcato per alcune architetture rispetto ad altre.

I risultati vanno letti alla luce di alcuni limiti dello studio. Su Oxford-IIIT Pet ciascuna configurazione è stata valutata su due soli seed, mentre su CIFAR-100 sono state addestrate due sole architetture, su un singolo seed, e la valutazione prototipica è stata condotta su un singolo episodio. Le percentuali di micro-drop non sono direttamente confrontabili tra i due dataset, poiché su Oxford-IIIT Pet sono calcolate sulle sole razze canine. Il set di validazione, utilizzato per l'early stopping, include anche le classi rimosse dal training, e l'F1-score sulle classi note risente in parte dei campioni delle classi rimosse, assegnati come falsi positivi alle classi note. Infine, la struttura gerarchica è sfruttata soltanto attra-

verso il backbone condiviso: le prestazioni della head *coarse* non sono state analizzate e non è stato effettuato un confronto con un classificatore *flat*.

6.2 Sviluppi futuri

Diversi aspetti del presente lavoro si prestano a ulteriori approfondimenti. Un primo passo naturale consiste nell'estendere a CIFAR-100 la stessa profondità sperimentale adottata per Oxford-IIIT Pet, addestrando le architetture mancanti e ripetendo la valutazione del modello prototipico su un numero di episodi comparabile, in modo da rendere il confronto tra i due dataset pienamente omogeneo.

Un secondo sviluppo riguarda l'aumento del numero di seed indipendenti per ciascuna configurazione, così da poter stimare con maggiore accuratezza la variabilità dei risultati e distinguere in modo più netto gli effetti sistematici da quelli dovuti al rumore di inizializzazione.

Infine, un'ulteriore direzione di ricerca potrebbe riguardare la valutazione del modello prototipico secondo lo scenario di *few-shot puro*, in cui la classificazione avviene esclusivamente tra le classi non viste, isolando così la capacità di apprendimento few-shot del modello dall'influenza delle classi già note; un confronto diretto tra questo scenario e quello *Generalized* adottato in questo lavoro permetterebbe di quantificare più precisamente il costo, in termini di prestazioni sulle classi note, dell'approccio qui proposto.

6.3 Considerazioni finali

Il presente elaborato ha affrontato un aspetto spesso trascurato nella valutazione dei modelli di classificazione: le loro prestazioni a fronte del degrado del dataset di training, distinguendo esplicitamente tra una riduzione uniforme degli esempi e una rimozione mirata di intere classi. I risultati ottenuti mostrano che queste due forme di degradazione hanno un impatto profondamente diverso sulle prestazioni, e che tale impatto non è uniforme tra le diverse architetture: EfficientNet ha ottenuto le prestazioni assolute più elevate in entrambi gli scenari testati, mentre un'architettura meno performante come ResNet18 ha mostrato una fragilità significativa sia in termini di prestazioni sia di stabilità.

L'adozione di un approccio a reti prototipiche ha inoltre dimostrato che è possibile recuperare, almeno parzialmente, la capacità di riconoscere le classi escluse dal training senza ricorrere a un nuovo addestramento completo del modello, sebbene con un costo non trascurabile in termini di accuratezza rispetto a quella ottenuta sulle classi apprese in modo

tradizionale. Questo risultato conferma il potenziale del few-shot learning come strumento pratico per scenari in cui il dataset di training è incompleto o soggetto ad aggiornamenti frequenti, una condizione tutt'altro che rara in contesti reali.

Nel complesso, lo studio condotto offre indicazioni utili sia dal punto di vista della scelta architetturale in presenza di dataset potenzialmente incompleti, sia come base metodologica per l'integrazione di tecniche di few-shot learning in pipeline di classificazione già esistenti.

Bibliografia

- [1] Kaiming He et al. «Deep Residual Learning for Image Recognition». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.
- [2] Gao Huang et al. «Densely Connected Convolutional Networks». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2017, pp. 4700–4708. DOI: 10.1109/CVPR.2017.243.
- [3] Alex Krizhevsky e Geoffrey Hinton. *Learning Multiple Layers of Features from Tiny Images*. Rapp. tecn. Toronto, Ontario: University of Toronto, 2009. URL: <https://www.cs.toronto.edu/~kriz/learning-features-2009-TR.pdf>.
- [4] Omkar M. Parkhi et al. «Cats and Dogs». In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 2012, pp. 3498–3505. DOI: 10.1109/CVPR.2012.6248092.
- [5] Jake Snell, Kevin Swersky e Richard Zemel. «Prototypical Networks for Few-Shot Learning». In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017, pp. 4077–4087.
- [6] Mingxing Tan e Quoc V. Le. «EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks». In: *Proceedings of the 36th International Conference on Machine Learning (ICML)*. Vol. 97. Proceedings of Machine Learning Research. 2019, pp. 6105–6114.
- [7] Jason Yosinski et al. «How Transferable Are Features in Deep Neural Networks?» In: *Advances in Neural Information Processing Systems*. Vol. 27. 2014, pp. 3320–3328.